Maven - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

MAVEN — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

---

TOPIC 1: PROJECT STRUCTURE

---

1. Introduction

1.1 What is it?

Maven is a BUILD AUTOMATION and DEPENDENCY MANAGEMENT tool for Java projects — it standardizes HOW a project is structured, HOW dependencies (like `spring-boot-starter-web`, seen throughout this course) are declared and downloaded, and HOW a project is compiled, tested, and packaged into a deployable artifact (`.jar`/`.war`).

1.2 Why was it introduced

Before Maven (and its predecessor, Ant), every Java project had its OWN ad-hoc folder layout and hand-written build scripts — a new developer joining ANY project had to first learn THAT project's specific, custom structure and build process. Maven fixes this with **"Convention over Configuration"** — a STANDARDIZED project layout and build lifecycle that every Maven project follows, so a developer familiar with Maven can immediately understand and build ANY Maven project, with ZERO project-specific learning required.

2. The Standard Directory Layout

```
my-project/
■■■ pom.xml <- the PROJECT OBJECT MODEL (full detail in Topic 2)
■■■ src/
■ ■■■ main/
■ ■ ■■■ java/ <- YOUR application source code
■ ■ ■ ■■■ com/company/app/
■ ■ ■ ■■■ Application.java
■ ■ ■■■ resources/ <- non-code files bundled into the artifact
■ ■ ■■■ application.properties
■ ■ ■■■ static/
■ ■■■ test/
■ ■■■ java/ <- YOUR test source code (JUnit tests)
■ ■ ■■■ com/company/app/
■ ■ ■■■ ApplicationTests.java
■ ■■■ resources/ <- test-specific resources (e.g., application-test.properties)
■■■ target/ <- BUILD OUTPUT (compiled classes, the final .jar) - NEVER commit this!
```

3. Real Life Analogy

- **Restaurant**: Maven's standardized project structure is like EVERY restaurant in a chain being required to organize their kitchen the SAME way — walk-in fridge always in the SAME relative spot, prep stations always laid out identically — so ANY chef trained at ONE location can walk into ANY OTHER location and immediately know exactly where everything is, with zero re-training needed.

4. Edge Cases

Case	Result
Placing source code OUTSIDE `src/main/java`	Maven won't automatically find/compile it, unless explicitly reconfigured — a common early mistake
Committing the `target/` directory to Git	Bloats the repository with generated, REPRODUCIBLE build artifacts — should always be excluded via `.gitignore`

5. Common Mistakes

- Committing the `target/` directory to version control — it's entirely regenerable and should be excluded.
- Deviating from the standard directory layout without a specific, deliberate reason — loses the "any Maven developer can immediately navigate this project" benefit.

6. Frequently Asked Interview Questions

1. **What is Maven?** A build automation and dependency management tool for Java projects, standardizing project structure and the build lifecycle.
2. **What does "Convention over Configuration" mean in Maven's context?** A standardized default project layout/behavior that requires NO explicit configuration to use, as long as you follow Maven's conventions — reducing the learning curve across different projects.
3. **What's in `src/main/java` vs `src/test/java`?** `src/main/java` holds your actual application source code; `src/test/java` holds your test code (JUnit tests), kept SEPARATE and excluded from the final production artifact.
4. **Should the `target/` directory be committed to version control?** No — it's entirely generated/reproducible build output and should be excluded via `.gitignore`.

TOPIC 2: POM (PROJECT OBJECT MODEL)

---

1. Introduction

What is it? The `pom.xml` file is the HEART of every Maven project — an XML file describing the project's identity, its dependencies, build configuration, and plugins. Every Maven command reads and acts based on this single file.

2. A Complete, Annotated `pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
<modelVersion>4.0.0</modelVersion>

<!-- PROJECT COORDINATES - uniquely identify this project in the world -->
<groupId>com.jobsradar</groupId> <!-- organization/company identifier -->
<artifactId>employee-management-api</artifactId> <!-- this specific project's name -->
<version>1.0.0</version> <!-- this project's version -->
<packaging>jar</packaging> <!-- output type: jar, war, pom -->

<!-- PARENT POM - inherits configuration (like Spring Boot's dependency version management) -->
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.2.0</version>
</parent>

<properties>
<java.version>17</java.version>
</properties>

<!-- DEPENDENCIES (full detail in Topic 3) -->
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>

<!-- BUILD PLUGINS (full detail in Topic 4) -->
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

3. The "Maven Coordinates" — GAV

```
groupId:artifactId:version (often abbreviated "GAV")

com.jobsradar : employee-management-api : 1.0.0
```

This THREE-PART identifier uniquely identifies EVERY artifact in the ENTIRE Maven ecosystem — no two different libraries can have the SAME GAV, exactly like how Java packages (Core Java, Topic 26) prevent naming collisions using reverse-domain-name conventions.

4. Real Life Analogy

- **Bank**: The `pom.xml` is like a bank ACCOUNT's master profile record — it uniquely identifies the account (GAV coordinates), lists all its linked services/products (dependencies), and defines the specific processing rules/procedures (plugins/build configuration) that apply to it.

5. Common Mistakes

- Manually managing dependency VERSIONS without a `<parent>` BOM (like `spring-boot-starter-parent`), risking version incompatibilities between related libraries (exactly the problem Spring Boot's Starter BOM, from the Spring Boot section, solves).
- Confusing `groupId` (organization) with `artifactId` (specific project name).

6. Frequently Asked Interview Questions

1 **What is `pom.xml`?** The Project Object Model — an XML file describing a Maven project's identity, dependencies, and build configuration.

2 **What are Maven "GAV coordinates"?** `groupId:artifactId:version` — the three-part identifier uniquely identifying every artifact across the entire Maven ecosystem.

3 **What does a `<parent>` POM provide?** Inherited configuration, including managed dependency VERSIONS (a BOM), avoiding manual version-compatibility management (as covered with Spring Boot's starters).

TOPIC 3: DEPENDENCIES

---

1. Introduction

What is it? Dependencies are EXTERNAL libraries your project needs (like `spring-boot-starter-web`, seen throughout this course) — Maven automatically DOWNLOADS them (from a remote repository, typically Maven Central) and adds them to your project's classpath, resolving even TRANSITIVE dependencies (dependencies OF your dependencies) automatically.

2. Declaring a Dependency

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
<version>3.2.0</version> <!-- often OMITTED when inherited from a parent BOM -->
<scope>compile</scope> <!-- DEFAULT scope, if omitted -->
</dependency>
```

3. Dependency Scopes — Full Reference

Scope	Available During	Included in Final Artifact?
`compile` (default)	Compilation, testing, AND runtime	Yes
`provided`	Compilation and testing, but NOT packaged (expected to be provided by the runtime environment, e.g., a Servlet API when deploying to an external app server)	No
`runtime`	Testing and runtime, NOT compilation (e.g., a JDBC driver — you code against `java.sql.Connection`, not the driver class directly)	Yes
`test`	ONLY testing (e.g., JUnit, Mockito)	No

4. Transitive Dependencies

```
YOUR PROJECT depends on -> spring-boot-starter-web
|
which ITSELF depends on -> spring-webmvc, tomcat-embed-core, jackson-databind, ...

Maven AUTOMATICALLY pulls in ALL of these transitive dependencies for you -
you never manually list tomcat-embed-core yourself!
```

5. Resolving Version Conflicts

If TWO different dependencies transitively require DIFFERENT versions of the SAME library, Maven uses "nearest wins" (the version DECLARED CLOSEST to your project in the dependency tree) by default — but you can EXPLICITLY override this:

```
<dependency>
<groupId>com.fasterxml.jackson.core</groupId>
<artifactId>jackson-databind</artifactId>
<version>2.16.0</version> <!-- EXPLICITLY pin this version, overriding transitive resolution -->
</dependency>

mvn dependency:tree # visualize the FULL dependency tree, including transitive dependencies - great
for debugging!
```

6. Real Life Analogy

- **Restaurant**: A dependency is like ordering a pre-made sauce from a supplier instead of making it from scratch — and that sauce's OWN ingredients list (transitive dependencies) automatically comes along with it, without you needing to separately source each individual ingredient yourself.

7. Common Mistakes

- Not understanding `mvn dependency:tree` exists, making version CONFLICTS (the notorious "dependency hell") much harder to diagnose.
- Manually specifying versions for EVERY dependency instead of relying on a `<parent>` BOM's managed versions, risking incompatible combinations.

8. Frequently Asked Interview Questions

1. **What is a "transitive dependency"?** A dependency OF one of your direct dependencies, automatically pulled in by Maven without you needing to declare it yourself.
2. **What are the four main dependency scopes?** `compile` (default, always available), `provided` (compile/test only, not packaged), `runtime` (test/runtime only, not compile-time), `test` (testing only).
3. **How does Maven resolve version conflicts between transitive dependencies by default?** "Nearest wins" — the version declared closest to your project in the dependency tree is used, though this can be explicitly overridden.
4. **What command shows the full dependency tree, useful for debugging version conflicts?** `mvn dependency:tree`.

TOPIC 4: PLUGINS

---

1. Introduction

What is it? Plugins are the ACTUAL WORKERS that perform Maven's build tasks — compiling code, running tests, packaging JARs. Maven's core itself does very little; almost EVERYTHING is implemented as a plugin, bound to specific phases of the Maven BUILD LIFECYCLE.

2. The Maven Build Lifecycle — Phases

```
validate -> compile -> test -> package -> verify -> install -> deploy

compile: compiles src/main/java (via the compiler plugin)
test: runs src/test/java tests (via the surefire plugin)
package: bundles everything into a .jar/.war (via the jar/war plugin)
install: copies the built artifact into your LOCAL Maven repository (~/.m2)
deploy: uploads the artifact to a REMOTE, shared repository
```

> Running `mvn install` AUTOMATICALLY runs EVERY phase BEFORE it too (validate, compile, test, package) — Maven phases are SEQUENTIAL and CUMULATIVE.

3. Key Plugins You've Already Been Using

```
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId> <!-- packages an EXECUTABLE jar with an embedded
server -->
</plugin>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<configuration>
<source>17</source>
<target>17</target>
</configuration>
</plugin>
```

4. Common Maven Commands

```
mvn compile # compiles source code
mvn test # runs tests
mvn package # builds the final .jar/.war
mvn clean # deletes the target/ directory (fresh start)
mvn clean install # FRESH build, running through ALL lifecycle phases, installing to your local repo
mvn spring-boot:run # runs the Spring Boot application directly via Maven (dev convenience)
```

5. Real Life Analogy

- **Restaurant**: Maven's LIFECYCLE is like the STANDARD sequence every dish goes through (prep → cook → plate → quality-check → serve) — PLUGINS are the SPECIFIC STATIONS/EQUIPMENT (the oven, the plating station) that actually PERFORM each step of that standardized sequence.

6. Frequently Asked Interview Questions

1. **What role do plugins play in Maven?** They're the actual workers performing build tasks (compiling, testing, packaging), bound to specific phases of Maven's build lifecycle.
2. **What are the main phases of the Maven build lifecycle, in order?** `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`.
3. **What does `mvn clean install` do?** Deletes the `target/` directory (fresh start), then runs through ALL lifecycle phases up to `install`, placing the built artifact in your local Maven repository.
4. **What plugin packages a Spring Boot application into an executable JAR with an embedded server?** `spring-boot-maven-plugin`.

TOPIC 5: PROFILES

---

1. Introduction

What is it? Maven Profiles let you define DIFFERENT build configurations (different dependencies, plugin settings, or properties) that activate under DIFFERENT conditions — CONCEPTUALLY similar to Spring Boot's application Profiles (Spring Boot section, Topic 6), but operating at the BUILD level, not the application-runtime level.

2. Code Example

```
<profiles>
<profile>
<id>dev</id>
<activation>
<activeByDefault>true</activeByDefault>
</activation>
<properties>
<db.url>jdbc:postgresql://localhost:5432/dev_db</db.url>
</properties>
</profile>

<profile>
<id>prod</id>
<properties>
<db.url>jdbc:postgresql://prod-server:5432/prod_db</db.url>
</properties>
<build>
<plugins>
<plugin>
<!-- e.g., run additional security/quality checks ONLY for production builds -->
</plugin>
</plugins>
</build>
</profile>
</profiles>
```

3. Activating a Profile

```
mvn clean install -P prod # activates the "prod" profile explicitly
```

4. Real Life Analogy

- **Restaurant**: Maven profiles are like a restaurant chain having a "TEST KITCHEN build" recipe configuration versus a "LIVE CUSTOMER-FACING build" recipe configuration — same overall recipe STRUCTURE, but DIFFERENT specific settings (ingredient sourcing, quality-check steps) depending on which mode is currently active.

5. Frequently Asked Interview Questions

1. **What are Maven Profiles used for?** Defining different build-time configurations (dependencies, properties, plugin behavior) that activate under different conditions.
2. **How do you explicitly activate a specific Maven profile?** `mvn <goal> -P <profile-id>`.

3 **How does a Maven Profile differ from a Spring Boot Profile?** Maven Profiles affect the BUILD process itself (compile-time configuration); Spring Boot Profiles affect the APPLICATION's runtime configuration/beans — conceptually similar goals, but operating at entirely different stages.

TOPIC 6: MULTI-MODULE PROJECTS

---

1. Introduction

What is it? A multi-module Maven project lets you organize a LARGE application as SEVERAL separate, independently-buildable sub-projects (modules) — e.g., splitting a system into `api`, `service`, `data-access`, and `common` modules — all managed under ONE parent `pom.xml`.

2. Structure

```
my-application/ <- PARENT project
■■■ pom.xml <- parent POM, lists all modules
■■■ common/ <- shared utility/DTO module
■ ■■■ pom.xml
■■■ data-access/ <- repository/entity module
■ ■■■ pom.xml
■■■ api/ <- REST controller module
■■■ pom.xml
```

3. Parent POM

```
<groupId>com.jobsradar</groupId>
<artifactId>my-application</artifactId>
<version>1.0.0</version>
<packaging>pom</packaging> <!-- 'pom' packaging - this is a PARENT/aggregator, not a deployable
artifact itself -->

<modules>
<module>common</module>
<module>data-access</module>
<module>api</module>
</modules>
```

4. A Child Module Depending on Another Module

```
<!-- api/pom.xml -->
<parent>
<groupId>com.jobsradar</groupId>
<artifactId>my-application</artifactId>
<version>1.0.0</version>
</parent>
<artifactId>api</artifactId>

<dependencies>
<dependency>
<groupId>com.jobsradar</groupId>
<artifactId>data-access</artifactId> <!-- depends on ANOTHER module in the SAME multi-module project
-->
<version>1.0.0</version>
</dependency>
</dependencies>
```

5. Building the Entire Project

```
mvn clean install # run from the PARENT directory - builds ALL modules, in the CORRECT dependency
order automatically
```

6. Real Life Analogy

- **Restaurant**: A multi-module Maven project is like a large restaurant CHAIN organized into separate but coordinated divisions (a central "supply chain" module, a "kitchen operations" module, a "front-of-house" module) — each can be worked on somewhat independently, but they're all managed and released together under ONE overarching company structure, with clear dependencies between them (front-of-house depends on kitchen operations, which depends on supply chain).

7. Edge Cases

Case	Result
A CIRCULAR dependency between two modules (module A depends on B, B depends on A)	Maven FAILS to build — a genuine design problem needing to be resolved by extracting shared code into a THIRD, lower-level module
Building a SINGLE child module in isolation, when it depends on ANOTHER unbuild child module	Fails unless the dependency has ALREADY been `mvn install`-ed to the local repository first

8. Common Mistakes

- Creating circular dependencies between modules — always a genuine architectural problem to fix, not a Maven limitation to work around.
- Trying to build a child module independently without first ensuring its module dependencies are already installed locally.

9. Frequently Asked Interview Questions

1. **What is a multi-module Maven project?** A project organized as several separate, independently-buildable sub-projects (modules), all coordinated under one parent POM.
2. **What packaging type does the PARENT POM use?** `pom` (not `jar`/`war`) — it's an aggregator, not a deployable artifact itself.
3. **What happens if two modules have a circular dependency on each other?** The build fails — this indicates a genuine design problem, typically resolved by extracting shared code into a separate, lower-level module both can depend on.
4. **How do you build ALL modules in a multi-module project at once, in the correct order?** Run `mvn clean install` from the PARENT directory — Maven automatically determines the correct build order based on inter-module dependencies.

10. Revision Notes (Entire Maven Section Consolidated)

```
MAVEN — CHEAT SHEET
=======================
"Convention over Configuration" - standardized project layout, everyone
immediately understands ANY Maven project
STRUCTURE: src/main/java (code), src/main/resources, src/test/java, target/
(NEVER commit target/)

POM.XML: groupId:artifactId:version (GAV) uniquely identifies a project
<parent> POM provides inherited/managed dependency versions (a BOM)

DEPENDENCIES: TRANSITIVE deps pulled in automatically
scopes: compile(default, everywhere) / provided(compile+test, NOT packaged) /
runtime(test+runtime, NOT compile) / test(testing only)
mvn dependency:tree -> visualize/debug version conflicts

BUILD LIFECYCLE (sequential, cumulative): validate -> compile -> test ->
package -> verify -> install -> deploy
PLUGINS do the actual work at each phase (compiler, surefire, spring-boot-maven-plugin)

PROFILES: different BUILD-time configs (dev/prod), activate via `mvn -P profile-id`
(different concept from Spring's runtime application profiles!)

MULTI-MODULE: parent POM (packaging=pom) + child modules, built together in
correct dependency order via `mvn clean install` from the parent
```

11. Homework (Covers Entire Maven Section)

1. Create a fresh Maven project from scratch, add `spring-boot-starter-web` as a dependency, and run `mvn dependency:tree` to see everything it pulls in transitively.
2. Add a `dev` and `prod` Maven profile with different property values, and build with each using `-P`.
3. Split a small existing project into a 2-module multi-module structure (`common` + `api`), with `api` depending on `common`, and build the whole thing with one `mvn clean install` from the parent.

FINAL SUMMARY — MAVEN COMPLETE

This concludes the **Maven** section — 6 topics:

1. **Project Structure** — the standardized layout, "Convention over Configuration."
2. **POM** — GAV coordinates, parent POMs and inherited version management.
3. **Dependencies** — transitive resolution, the four scopes, conflict resolution.
4. **Plugins** — the build lifecycle phases, and how plugins do the actual work.
5. **Profiles** — build-time configuration switching (vs Spring's runtime profiles).
6. **Multi-module Projects** — organizing large applications into coordinated sub-projects.

Every dependency you've added throughout this entire course (Spring Boot starters, Hibernate, JWT libraries, PostgreSQL drivers) has gone through exactly this Maven mechanism. Next: Git, covering the version control system managing all this code.

(END OF MAVEN MASTER NOTES — ready to proceed to Git whenever you say "Next Topic.")